

Fundamentos da Qualidade e Produtividade

01

2026

SENAI - CRTI



Princípios e Gestão da Qualidade

Unidade Curricular: Fundamentos da Qualidade e Produtividade

Vamos conectar teoria da qualidade com a prática real do desenvolvimento de software — da inspeção ao produto que encanta o usuário.

O que vamos percorrer hoje?

Esta aula está estruturada em seis momentos pedagógicos, seguindo a lógica da **Metodologia MSEP**. Cada etapa foi desenhada para construir conhecimento de forma progressiva, sempre conectando teoria com prática de desenvolvimento de software.

01

Situação-Problema

Contextualização com um cenário realista de falha de qualidade

02

Mobilização de Conhecimentos

Reflexões sobre o que já sabemos sobre qualidade

03

Desenvolvimento Teórico

Conceitos, pilares e evolução da gestão da qualidade

04

Aplicação Prática

Análise guiada de sistema fictício com problemas reais

05

Sistematização

Resumo visual dos conceitos aprendidos

06

Atividade Final

Trabalho em grupo com ferramentas reais do mercado

Um Cenário Que Todo Desenvolvedor Conhece

O que aconteceu?

A equipe da **Startup TechNow** passou 6 meses desenvolvendo um sistema de gestão para uma rede de clínicas. Na data de entrega, o cliente recebeu o software — e o caos começou.

- Pacientes não conseguiam agendar consultas online
- Dados de prontuários sumiam após salvar
- A interface travava em celulares
- Relatórios geravam valores incorretos
- Nenhum teste automatizado havia sido criado

As consequências

- Retrabalho: 3 meses de correções emergenciais
- Custo dobrou: horas extras não planejadas
- Cliente insatisfeito: pediu desconto e ameaçou cancelar contrato
- Equipe desmotivada: estresse e sobrecarga
- Reputação da empresa comprometida

📌 **Problema central:** Falta de qualidade no *processo* e no *produto* — não apenas nos bugs, mas na forma como o software foi construído.

Quem Sofre no Final?

Quando a qualidade falha, o impacto não fica só dentro da equipe de desenvolvimento. Ele se propaga — e sempre chega ao usuário final.



O Usuário

Frustração, perda de confiança no sistema e abandono da plataforma

O Cliente

Prejuízo financeiro, operação comprometida, insatisfação com o fornecedor

A Equipe

Retrabalho, horas extras, desmotivação e pressão constante

O Negócio

Custos elevados, perda de contratos e danos à reputação da empresa

O Que Você Já Sabe Sobre Qualidade?

Antes de apresentar os conceitos formais, vamos ativar o que você já vivenciou. Reflita individualmente e, depois, compartilhe com a turma. Não existe resposta errada — o objetivo é **pensar criticamente** sobre o que é qualidade para você.

Pergunta 1

O que é um software "de qualidade"? Quais características um sistema precisa ter para você dizer que ele é bom?

Pergunta 2

Já usou um sistema ruim? Qual era o problema? Travava? Perdia dados? Interface confusa? O que mais te incomodava?

Pergunta 3

Qual a diferença entre "funcionar" e "ter qualidade"? Um sistema pode funcionar e ainda assim ser de baixa qualidade?

Pergunta 4

Quem define se um software tem qualidade? O desenvolvedor? O gerente? O cliente? O usuário?

"Funcionar" vs. "Ter Qualidade"

Essa distinção é fundamental. Um software pode executar suas funções básicas e ainda assim falhar em atender às reais necessidades do usuário. Veja a diferença na prática:

☒ Apenas "Funciona"

- O botão de salvar grava os dados
- O sistema não cai durante o uso
- O login autentica o usuário
- O relatório é gerado
- O código passa na compilação

★ Tem Qualidade

- Salva em menos de 1 segundo, com feedback visual
- Suporta 1.000 usuários simultâneos sem lentidão
- Login seguro com autenticação em dois fatores
- Relatório intuitivo, exportável e com dados corretos
- Código legível, testado e documentado

 **Definição prática:** Qualidade é a *totalidade de características* que satisfazem necessidades declaradas e implícitas do usuário — não apenas a ausência de erros.

A Evolução do Conceito de Qualidade

A qualidade não surgiu como a conhecemos hoje. Ela evoluiu ao longo do século XX, passando por quatro grandes eras — cada uma com foco diferente, e cada uma ainda presente no desenvolvimento de software moderno.



Cada era representou um avanço na forma de pensar qualidade: do simples "checar se está certo" até "criar uma cultura organizacional que previne erros e busca excelência continuamente".

Era 1: Inspeção

Período: Início do século XX — industrialização em massa. O foco era simples: inspecionar o produto *depois* de pronto e separar os defeituosos dos bons.

Como funcionava

- Inspetor verificava cada item ao final da linha de produção
- Produtos com defeito eram descartados ou refeitos
- Qualidade era responsabilidade de uma única pessoa
- Processo custoso: defeitos descobertos tarde

Analogia com Software

- Testar o sistema somente ao final do projeto
- Encontrar bugs apenas na entrega ao cliente
- QA como "gargalo" no final do pipeline
- Retrabalho caro e desmotivante para a equipe



Era 2: Controle Estatístico da Qualidade

A grande virada

Na década de 1930, Walter Shewhart e W. Edwards Deming introduziram métodos estatísticos para **monitorar os processos** — não apenas o produto final. O objetivo mudou: de detectar para prevenir.

- Gráficos de controle para identificar variações
- Coleta de dados durante a produção
- Causas dos defeitos investigadas

No Desenvolvimento de Software

- **Métricas de processo:** taxa de bugs por sprint, tempo médio de resolução
- **Testes contínuos:** testes unitários e de integração ao longo do desenvolvimento
- **Code review:** revisão de código como controle preventivo
- **Dashboards de qualidade:** SonarQube, cobertura de testes

📋 **Princípio-chave:** É mais barato prevenir um bug do que corrigi-lo após a entrega — regra "1-10-100" do custo da qualidade.

Era 3: Garantia da Qualidade

A partir dos anos 1950-60, surgiu a necessidade de **sistematizar** a qualidade por meio de normas, processos documentados e auditorias. Qualidade passou a ser planejada, não apenas controlada.



Normas e Padrões

Criação de normas internacionais como a **ISO 9001**. Em software: **ISO/IEC 25010** (qualidade de produto) e **CMMI** (maturidade de processo)



Auditorias de Processo

Avaliações internas e externas para verificar se os processos estão sendo seguidos. Em TI: auditorias de segurança, pentest, conformidade com LGPD



Processos Documentados

Definição formal de como o trabalho deve ser feito. Em software: **gitflow**, política de branches, definição de "done" no Scrum

Era 4: Gestão da Qualidade Total (TQM)

A partir dos anos 1980, a qualidade deixou de ser responsabilidade de um departamento e passou a ser **cultura organizacional**. Todos — da direção ao estagiário — são responsáveis pela qualidade.



Todos Envolvidos

Cada membro da equipe contribui para a qualidade do produto



Foco no Cliente

O usuário final define o que é qualidade, não apenas o requisito técnico



Melhoria Contínua

Aprendizado constante, feedback e ciclos iterativos de evolução



Decisões por Dados

Métricas e evidências orientam as decisões, não suposições



Os 4 Pilares da Gestão da Qualidade

A norma **ISO 9001** e os principais frameworks modernos de qualidade se apoiam em quatro pilares fundamentais. No contexto de desenvolvimento de software, cada pilar tem aplicações diretas e práticas que você usará no dia a dia profissional.



1. FOCO NO CLIENTE

Entender e superar as necessidades do usuário final



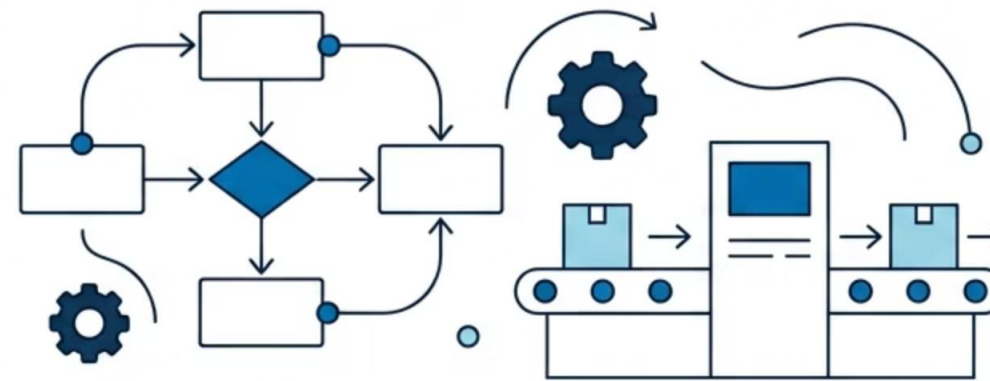
2. LIDERANÇA

Criar ambiente de propósito e direção clara para a equipe



3. ENGAJAMENTO DAS PESSOAS

Toda a equipe comprometida com a qualidade



4. ABORDAGEM POR PROCESSOS

Atividades gerenciadas como processos interligados

Pilar 1: Foco no Cliente

O cliente — e principalmente o **usuário final** — é a razão de existir de qualquer software. Qualidade não é o que o desenvolvedor acha que é bom: é o que resolve o problema real de quem usa.

Na Teoria

- Entender requisitos explícitos e implícitos
- Superar expectativas — não apenas atendê-las
- Medir e monitorar a satisfação continuamente
- Transformar feedback em ação de melhoria

Na Prática (TI)

- **UX Research:** entrevistas, testes de usabilidade
- **NPS e CSAT:** métricas de satisfação do usuário
- **Analytics:** onde o usuário clica, onde abandona
- **User stories:** requisitos escritos na perspectiva do usuário



Pilar 2: Liderança

Qualidade não acontece por acaso — ela é **construída intencionalmente** pela liderança da equipe. Um bom líder técnico cria o ambiente, os processos e a cultura necessários para que qualidade seja possível.

Scrum Master

Remove impedimentos, facilita cerimônias, garante que o processo ágil seja seguido com rigor e consistência.

Tech Lead

Define padrões de código, conduz code reviews, promove boas práticas (SOLID, Clean Code) e orienta o time técnico.

Product Owner

Prioriza o backlog com foco no valor para o usuário, garante que a equipe construa o produto certo com qualidade.

Cultura Psicológica Segura

Líderes eficazes criam ambiente onde erros são aprendizados e qualidade é responsabilidade coletiva, sem punição.

Pilar 3: Engajamento das Pessoas

Qualidade é um esporte coletivo. Quando toda a equipe está comprometida — e não apenas o QA — o resultado é exponencialmente melhor. O **engajamento genuíno** transforma uma equipe comum em uma equipe de alta performance.

Pair Programming

Dois desenvolvedores trabalham juntos: um escreve código, o outro revisa em tempo real

Code Review

Todo código é revisado por outro membro antes do merge — evita bugs e compartilha conhecimento

Definition of Done

A equipe define critérios claros do que significa "terminado" — incluindo testes, documentação e revisão

Retrospectivas

Toda sprint termina com reflexão coletiva: o que foi bem, o que pode melhorar, o que faremos diferente



Pilar 4: Abordagem por Processos

Qualidade consistente nasce de **processos bem definidos**, não de heróis individuais. Quando o processo é bom, o resultado bom é previsível e repetível.

Processos de Desenvolvimento

- **Gitflow:** controle de versão estruturado (branches main, develop, feature, hotfix)
- **CI/CD Pipeline:** integração e entrega contínua automatizada
- **Padrões de código:** linters, formatters, guias de estilo
- **Testes automatizados:** unitários, de integração, end-to-end

Exemplo de Pipeline de Qualidade

- 1. Developer faz commit
- 2. Linter verifica padrão de código
- 3. Testes unitários executam automaticamente
- 4. Code review obrigatório por par
- 5. Testes de integração em ambiente staging
- 6. Deploy automatizado após aprovação

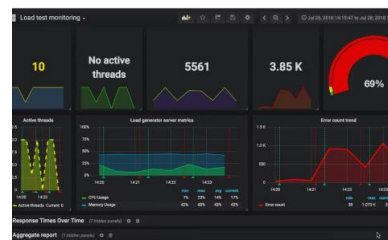
Não Decida no Escuro: Use Dados

Um dos maiores erros em desenvolvimento de software é tomar decisões baseadas em **suposições, opiniões ou "achismo"**. A gestão da qualidade exige que as decisões sejam fundamentadas em métricas e evidências concretas.



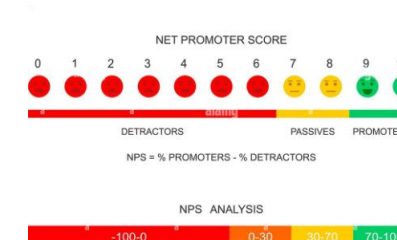
Métricas de Produto

- Taxa de bugs por sprint
- Densidade de defeitos (bugs/KLOC)
- Cobertura de testes (%)
- Débito técnico acumulado



Métricas de Performance

- Tempo de resposta da API
- Uptime e disponibilidade (%)
- Tempo de carregamento (Core Web Vitals)
- Taxa de erros em produção



Métricas de Satisfação

- NPS (Net Promoter Score)
- CSAT (Customer Satisfaction)
- Taxa de abandono (Churn)
- Tickets de suporte abertos

Da Suposição à Evidência

Veja como a mudança de mentalidade transforma a qualidade das decisões em projetos reais:

✗ Decisão por Suposição

"Acho que a tela de checkout está boa"

"O sistema está performando bem"

"Os usuários estão satisfeitos"

"Não temos muitos bugs"

"O código está bem escrito"

☑ Decisão por Evidência

"Temos 65% de abandono no passo 3 — analisamos via analytics"

"Latência média: 450ms — acima do SLA de 200ms definido"

"NPS: 32 — abaixo da meta de 50. Feedback aponta lentidão no mobile"

"84 bugs abertos, sendo 12 críticos — densidade 3x acima da meta"

"SonarQube aponta 47% de cobertura — meta é 80%"

O Ciclo PDCA: A Engrenagem da Melhoria

O ciclo **PDCA** (Plan-Do-Check-Act), desenvolvido por Deming, é a base filosófica da melhoria contínua. Em desenvolvimento ágil, ele se manifesta naturalmente a cada sprint — tornando o time progressivamente melhor.

Plan — Planejar

Identificar problema ou oportunidade de melhoria. Definir metas, hipóteses e plano de ação. Ex: backlog de débito técnico

Act — Agir

Se funcionou: padronizar e expandir. Se não: replanejar. Ex: adicionar novo padrão ao guia de desenvolvimento



Do — Executar

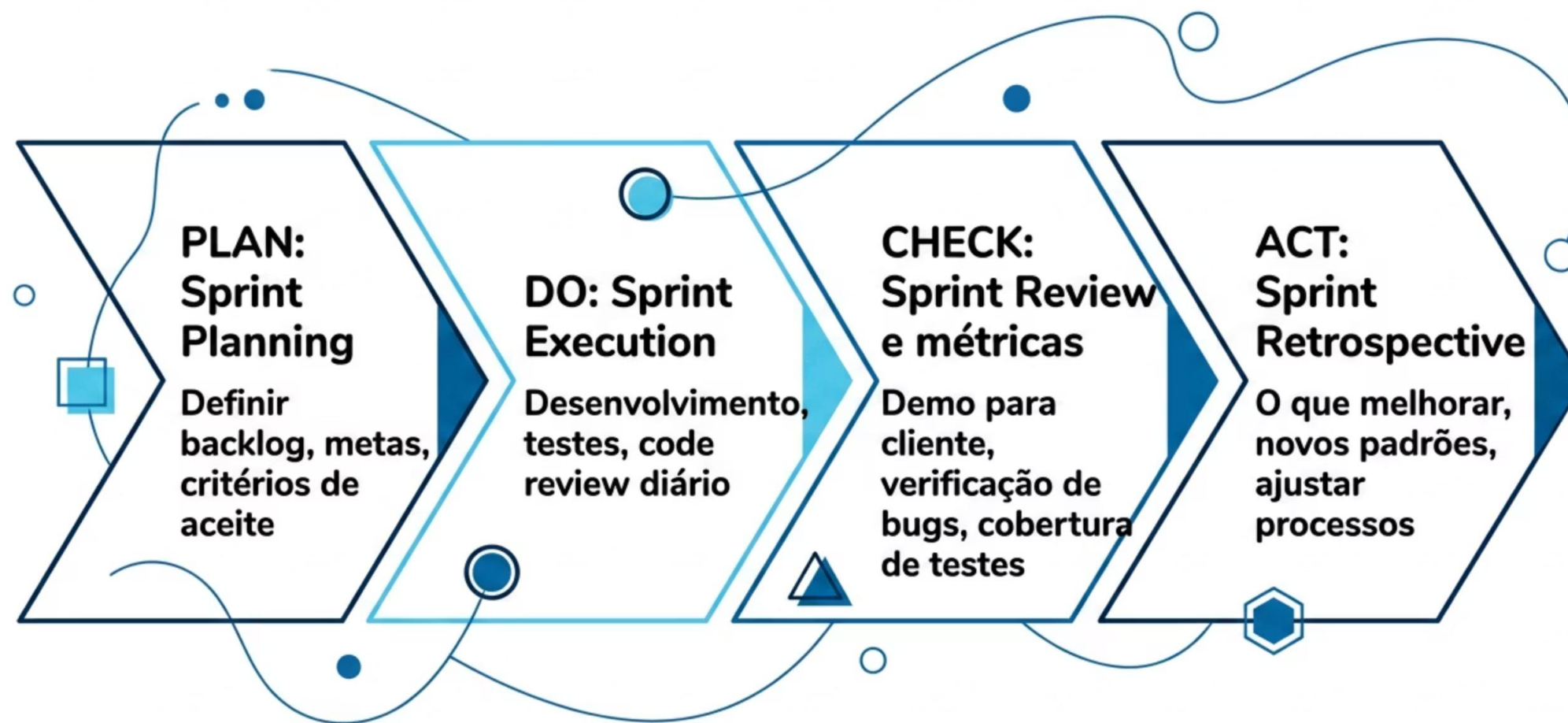
Implementar a melhoria em escala controlada. Ex: refatorar módulo de pagamento em uma sprint específica

Check — Verificar

Medir resultados com dados. A melhoria funcionou? Ex: comparar taxa de bugs antes e depois da refatoração

PDCA na Prática: Uma Sprint Ágil

Observe como o PDCA se encaixa naturalmente no ritmo de uma sprint de desenvolvimento ágil — transformando cada iteração em um ciclo de aprendizado e melhoria:



❏ **Insight:** Times ágeis que fazem retrospectivas sérias com dados — e realmente agem nas melhorias levantadas — são equipes que praticam PDCA naturalmente, mesmo sem saber.

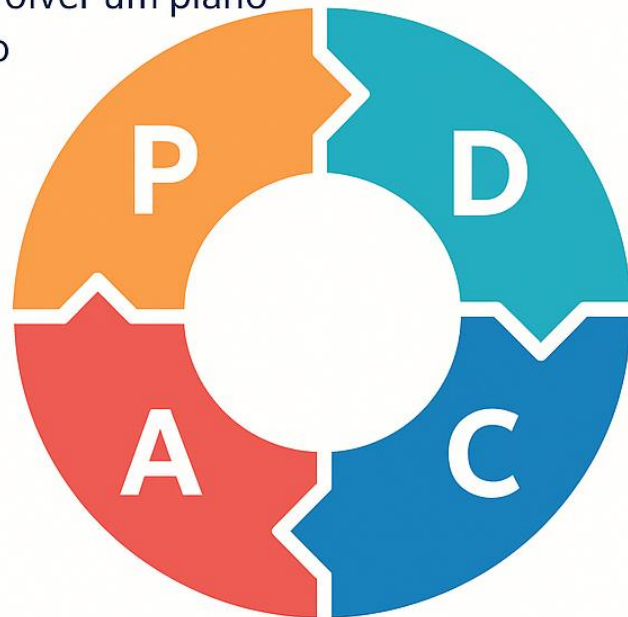
CICLO PDCA

PLAN – PLANEJAR

- ✓ Identificar o problema
- ✓ Analisar e encontrar a causa
- ✓ Desenvolver um plano de ação

DO – EXECUTAR

- ✓ Implementar a solução
- ✓ Executar o plano de ação



ACT – AGIR

- ✓ Padronizar a solução
- ✓ Agir corretivamente, se necessário

CHECK – VERIFICAR

- ✓ Avaliar os resultados
- ✓ Verificar se houve melhoria

Retrospectivas Ágeis: PDCA em Ação

A retrospectiva é o momento mais poderoso da metodologia ágil para promover melhoria contínua. Mas ela só funciona quando conduzida com **honestidade, dados e comprometimento com a mudança**.

1

O que foi bem?

Identifique práticas que geraram qualidade e devem ser mantidas na próxima sprint

2

O que pode melhorar?

Levante problemas com evidências — bugs, atrasos, comunicação falha

3

O que faremos diferente?

Defina ações concretas, com responsável e prazo — não apenas intenções vagas

4

Verificação

Na próxima retrospectiva, verifique se as ações foram cumpridas e se trouxeram melhoria real

Qualidade de Software: Uma Visão Completa

A norma **ISO/IEC 25010** define as características que um software de qualidade deve possuir. Conhecer essas dimensões é essencial para avaliar e construir sistemas melhores.

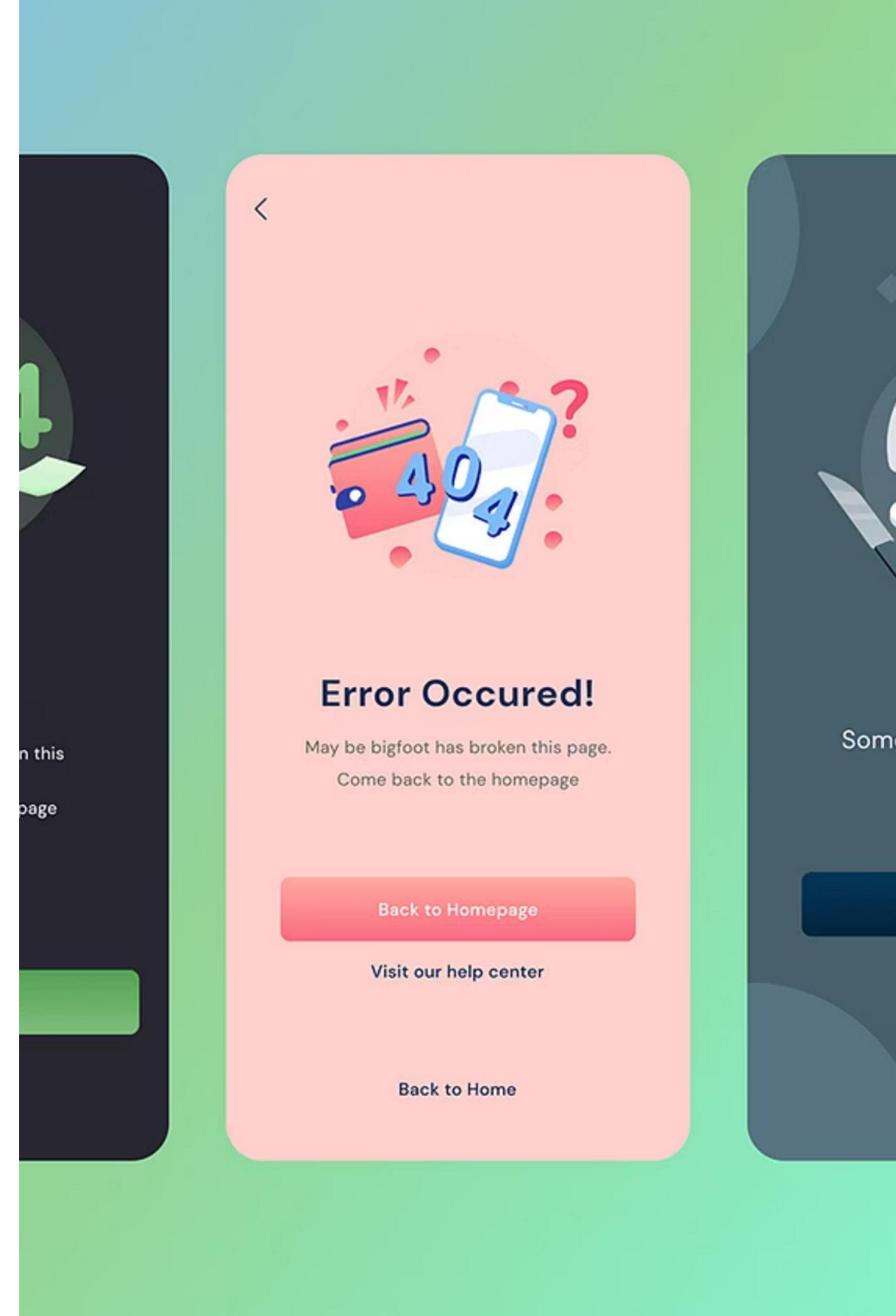


PARTE 4 — APLICAÇÃO PRÁTICA GUIADA

Atividade Guiada: Diagnóstico do Sistema "ClinicaFácil"

Chegou a hora de colocar a teoria em prática. Você vai analisar o sistema fictício "**ClinicaFácil**" — o mesmo sistema da Startup TechNow que vimos no início — e aplicar os conhecimentos da gestão da qualidade.

 **Instruções:** Trabalhem em grupos de 3 a 4 pessoas. Cada grupo analisa o sistema pelos 4 pilares da qualidade.



Problemas Identificados no ClinicaFácil

Analise cada problema abaixo e tente relacionar com um pilar da qualidade (ou mais de um). Depois, proponha uma melhoria concreta:

#	Problema Identificado	Pilar(es) Violado(s)	Melhoria Sugerida
1	Interface não funciona em celulares — 70% dos médicos acessam pelo smartphone	Foco no Cliente	Sua sugestão aqui...
2	Não há testes automatizados — todos os testes eram manuais e incompletos	Abordagem por Processos	Sua sugestão aqui...
3	Nenhum code review era feito — cada dev subia código direto para produção	Liderança / Engajamento	Sua sugestão aqui...
4	Métricas de uso nunca foram coletadas — problemas descobertos só por reclamações	Decisão por Evidências	Sua sugestão aqui...
5	Sem retrospectivas — erros se repetiam sprint após sprint sem nenhuma correção	Melhoria Contínua	Sua sugestão aqui...

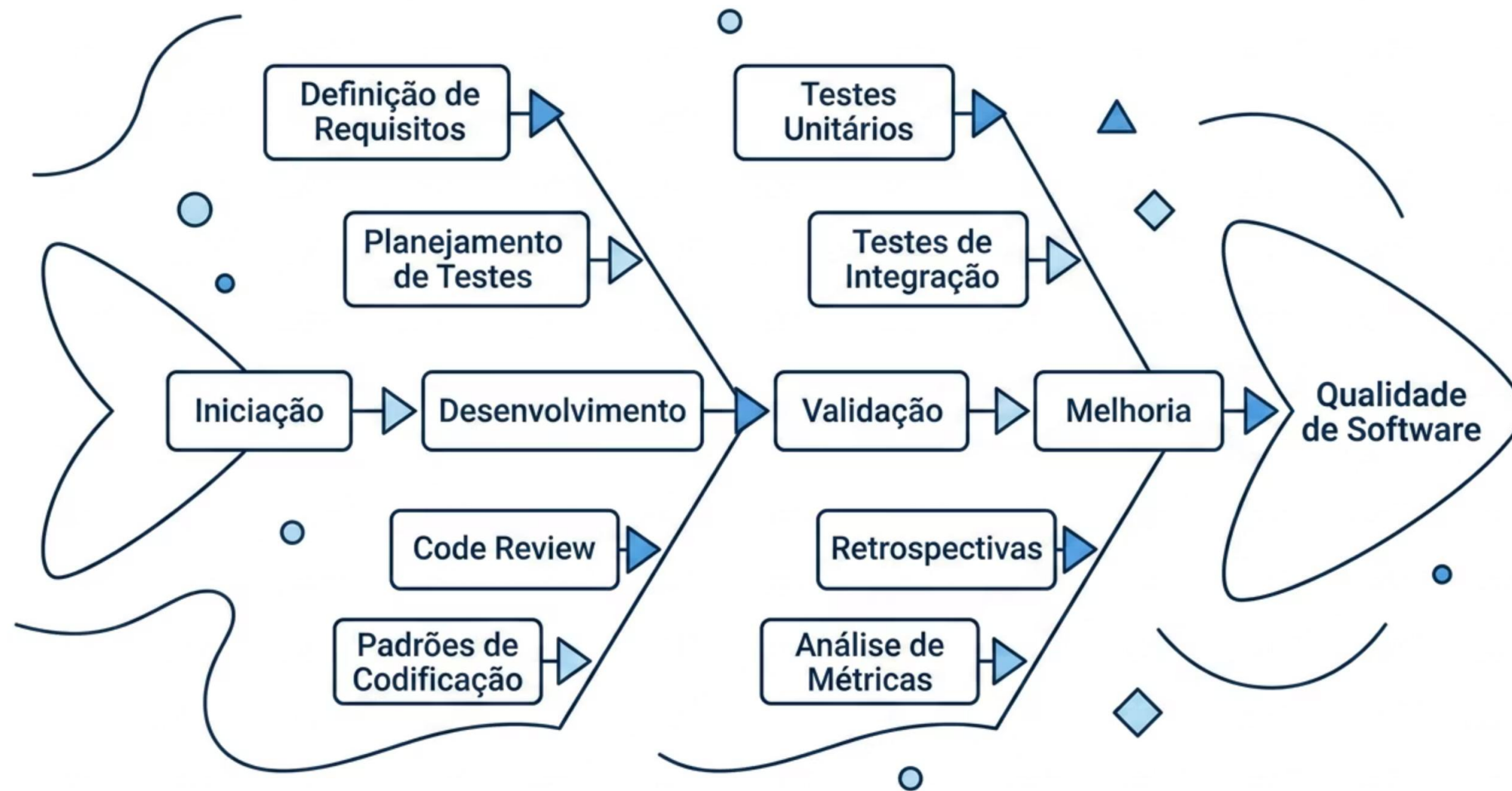
Propondo Melhorias para o ClinicaFácil

Com base na análise, seu grupo deve propor um **Plano de Melhoria** estruturado nos pilares. Use a tabela abaixo como guia para estruturar as propostas:

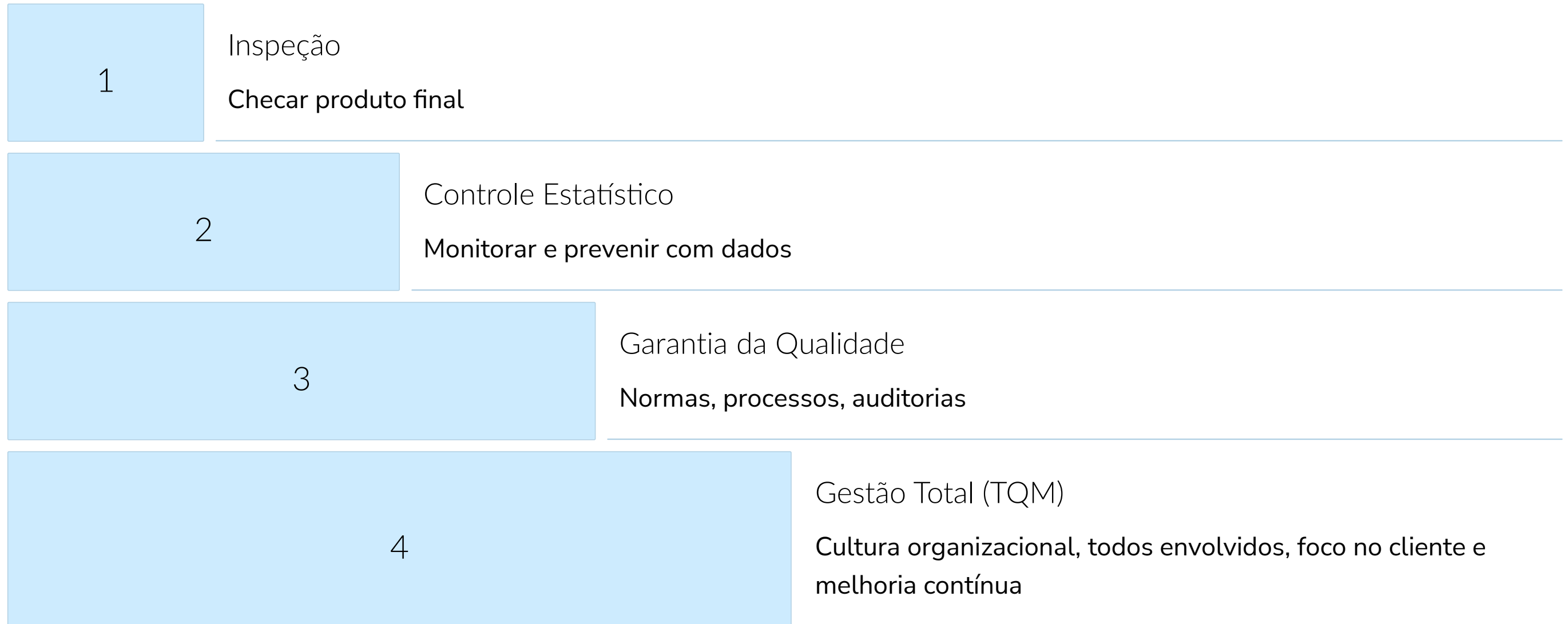
Foco no Cliente Ação: Realizar teste de usabilidade com 5 médicos antes do próximo release Métrica: Taxa de conclusão de tarefas > 90% e SUS Score > 70	Liderança Ação: Tech Lead define coding standards e torna code review obrigatório via pull request Métrica: 100% dos PRs revisados antes do merge
Engajamento Ação: Implementar pair programming nos módulos críticos e retrospectivas quinzenais Métrica: Redução de 50% em bugs críticos por sprint	Processos Ação: Criar pipeline CI/CD com testes automatizados e cobertura mínima de 70% Métrica: 0 deploys sem passar pelo pipeline de qualidade

Mapa da Qualidade: Tudo Conectado

Veja como os conceitos aprendidos hoje se relacionam e formam um sistema coeso de gestão da qualidade em desenvolvimento de software:



Resumo: Evolução da Qualidade



Em software, essas eras coexistem: você usa inspeção (code review), controle (métricas de bugs), garantia (CI/CD, ISO) e gestão total (cultura DevOps e ágil) — **ao mesmo tempo**.

Resumo: Os 4 Pilares + Ferramentas de TI

Pilar	Princípio	Ferramentas e Práticas em TI
Foco no Cliente	Entender e superar necessidades do usuário final	UX Research, NPS, User Stories, Analytics (GA, Hotjar)
Liderança	Criar ambiente e processos que habilitam qualidade	Tech Lead, Scrum Master, Coding Standards, Pull Requests
Engajamento	Toda equipe responsável pela qualidade	Pair Programming, Code Review, Definition of Done, Retrospectivas
Processos	Processos bem definidos geram resultados consistentes	Gitflow, CI/CD, Testes Automatizados, SonarQube, JIRA

 **Lembre-se:** Os pilares são interdependentes. Processos sem liderança se tornam burocracia. Foco no cliente sem processos gera caos. A força está na combinação de todos.

Resumo: O Ciclo de Melhoria Contínua

Ciclo PDCA Aplicado

1 Plan

Sprint Planning + definição de critérios de qualidade

2 Do

Desenvolvimento com TDD, code review e pair programming

3 Check

Sprint Review + análise de métricas e cobertura de testes

4 Act

Retrospectiva + atualização de processos e padrões

Métricas Essenciais de Qualidade

- **Taxa de bugs por sprint** — tendência deve cair
- **Cobertura de testes** — meta mínima 70-80%
- **Tempo médio de resolução** — bugs críticos <24h
- **NPS / CSAT** — satisfação do usuário
- **Lead time** — do commit ao deploy
- **Débito técnico** — monitorar e reduzir

Atividade Final: Auditoria de Qualidade em App Real

Agora é a hora de aplicar tudo que aprendemos em um contexto real. Em grupos de 3 a 5 pessoas, escolham **uma ferramenta consolidada no mercado** e realizem uma auditoria de qualidade.

 iFood

App de delivery: explorar fluxo de pedido, rastreamento, notificações e suporte

 Mercado Livre

E-commerce: busca, checkout, pagamento, rastreamento de entrega

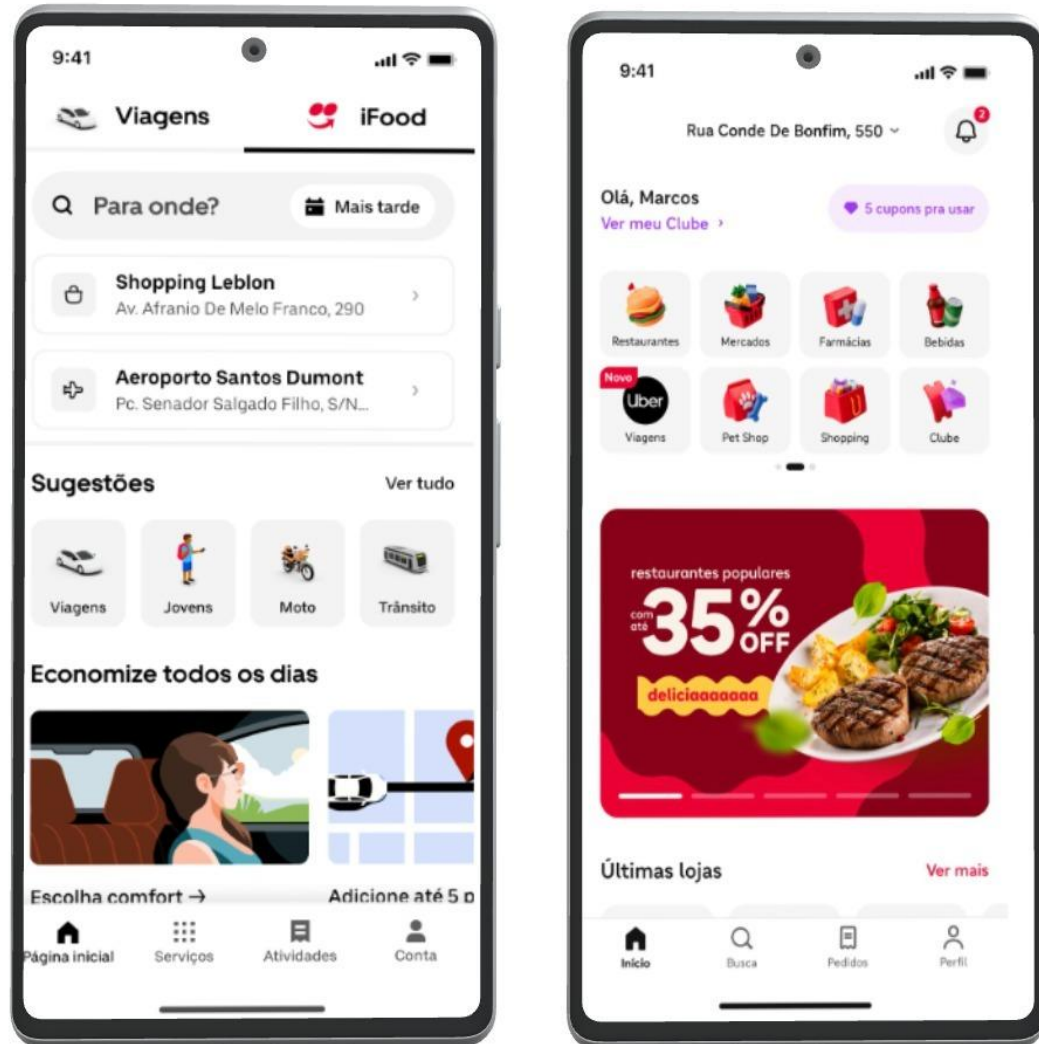
 Rede Social

Instagram, TikTok ou X: feed, busca, notificações, configurações de privacidade

 Banco Digital

Nubank, Inter ou C6: transferência, pagamentos, investimentos, suporte

Duração: 30 minutos — 20 min para análise em grupo + 10 min para apresentação e debate em turma.



Roteiro da Atividade Final

Siga as etapas abaixo para estruturar a análise do grupo. Ao final, prepare uma apresentação de **5 minutos** para a turma — como se fossem consultores de qualidade apresentando seus achados a um cliente.



Etapa 1: Identificação de Problemas

Use o app por 5-10 minutos. Anote tudo que incomoda: lentidão, confusão na interface, bugs, fluxos quebrados, mensagens de erro ruins



Etapa 2: Relacionar com os Pilares

Para cada problema encontrado, identifique qual pilar da qualidade foi violado e qual possível falha de processo o originou



Etapa 3: Propor Melhorias

Sugira melhorias práticas e realistas — não apenas "refazer tudo". Pense em quick wins e melhorias estruturais de médio prazo



Etapa 4: Definir Métricas

Para cada melhoria proposta, defina uma métrica que permita acompanhar se a melhoria realmente funcionou. Ex: taxa de abandono, NPS, tempo de carregamento

Conclusão: Qualidade é uma Escolha Diária

Qualidade em software não nasce de uma decisão única — ela é construída **sprint após sprint, commit após commit, decisão após decisão**. Todo desenvolvedor, designer, analista e gestor de TI é responsável por ela.

Qualidade é cultura, não departamento

Não existe "time de qualidade" responsável por algo que é de toda a equipe. QA é aliado, não inspetor final.

O usuário tem sempre a palavra final

Não importa o quão tecnicamente perfeito seja o código — se o usuário sofre para usá-lo, a qualidade falhou.

Prevenir é mais barato que corrigir

Investir em testes, processos e boas práticas hoje evita retrabalho caro e clientes insatisfeitos amanhã.

Dados iluminam o caminho

Meça, analise, decida — e repita. A melhoria contínua é um ciclo, não um destino.

Capacidades Desenvolvidas Hoje

Ao final desta aula, você deve ser capaz de:



Compreender a Evolução

Explicar as quatro eras da qualidade e como cada uma se manifesta no desenvolvimento de software atual



Aplicar os Pilares

Relacionar os 4 pilares da gestão da qualidade com práticas concretas de TI (CI/CD, UX, Scrum)



Decidir por Evidências

Identificar e utilizar métricas de qualidade para embasar decisões técnicas e de produto



Promover Melhoria

Aplicar o ciclo PDCA e retrospectivas ágeis como ferramentas de evolução contínua em projetos de TI

